Multiscale Progressive Segmentation Network for High-Resolution Remote Sensing Imagery

Renlong Hang, *Member, IEEE*, Ping Yang, *Graduate Student Member, IEEE*, Feng Zhou, *Member, IEEE*, and Qingshan Liu, *Senior Member, IEEE*

***Abstract*— Semantic segmentation of high-resolution remote sensing imageries (HRSIs) is a critical task for a wide range of applications, such as precision agriculture and urban planning. Although convolutional neural networks (CNNs) have made great progress in accomplishing this task recently, there still exist some challenges to address, one of which is simultaneously segmenting objects with large-scale variations in a HRSI. Targeting at this challenge, previous CNNs often adopt multiple convolution kernels in one layer or skip-layer connections between different layers to extract multiscale representations. However, due to the limited learning capacity of each CNN, it tends to make tradeoffs in segmenting different-scale objects. This would lead to unsatisfactory segmentation results for some objects, especially the small or the large ones. In this article, we propose a multiscale progressive segmentation network to address this issue. Instead of forcing one network to deal with all scales of objects, our network attempts to cascade three subnetworks for gradually segmenting objects into small scale, large scale, and other scale. In order to make the subnetwork focus on the specific scale objects, a scale guidance module is designed. It takes advantage of segmentation results from the preceding subnetwork to guide the feature learning of the succeeding one. Additionally, to acquire the final segmentation results, we propose a position-sensitive module for adaptively combining the outputs of the three subnetworks. This module is capable of assigning combination weights of different subnetworks according to their importance. Experiments on two benchmark datasets named Vaihingen and Potsdam indicate that our proposed network can achieve considerable improvements in comparison with several state-of-the-art segmentation models.**

***Index Terms*— Convolutional neural networks, high-resolution remote sensing imagery, multiscale objects, position-sensitive modules, scale guidance modules, semantic segmentation.**

I. Introduction

SEMANTIC segmentation is a long-standing topic in the field of remote sensing imagery interpretation and understanding [1], [2], [3]. Its primary purpose is to recognize the category of each pixel in the imagery. Recently, with the rapid development of imaging and sensor technologies, high-resolution remote sensing imageries (HRSIs) are now easier to acquire than before, making it possible to capture detailed information over regions of interest. However, HRSI also brings some challenges for accurate object segmentation. For instance, the variations within the same class of objects would be increased, while the diversity between different classes of objects would be reduced, which poses difficulties in learning robust and discriminative feature representations.

Recent years have witnessed the successful applications of convolutional neural networks (CNNs) to learn high-level semantic features from natural images [4]. Therefore, CNN becomes an intuitive choice to address the aforementioned issue. Early attempts along this pipeline were dependent on patch models [5], [6], [7], [8]. Typically, for each pixel, they first crop an image patch centered at it. Then, several convolutional layers are employed to extract spatial features from the cropped patch. Finally, a few fully connected layers are used to derive the classification result. Although such models could improve the segmentation performance when compared to traditional models with hand-crafted features, they have at least two drawbacks. The first one is that patch-wise classifications cost much time to finish segmenting the whole HRSI, which often contains millions of pixels. The second one is that CNNs can only extract features from local patches while ignoring the discriminative information outside that scope. In 2015, Long *et al.* proposed a fully convolutional network (FCN) for accurate and efficient natural image segmentation [9]. It replaces the fully connected layers in traditional CNNs with convolution and upsampling operators so that the image-to-image segmentation task can be implemented in an end-to-end manner. Inspired by it, a lot of works extended FCN to segment HRSIs [3], [10] and achieved better performance than patch-based CNNs.

In comparison with natural images, HRSIs have much larger sizes due to the large coverage area. Take the Potsdam data shared by International Society for Photogrammetry and Remote Sensing (ISPRS) as an example, the spatial size of each image is 6000×6000 pixels. They often cover various types of objects, whose scales range a lot, from very small ones like cars to very large ones like large buildings. In terms of FCN structures, objects with different scales have different requirements. Specifically, deeper networks usually provide larger receptive fields, which are beneficial to extract enough context information for large-scale object segmentation.

Manuscript received 16 June 2022; revised 18 August 2022; accepted 13 September 2022. Date of publication 16 September 2022; date of current version 3 October 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0112200 and in part by the Natural Science Foundation of China under Grant 61906096 and Grant U21B2044. *(Corresponding author: Qingshan Liu.)*

The authors are with the Engineering Research Center of Digital Forensics, Ministry of Education, Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology, School of Computer and Software, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: renlong_hang@163.com; pingyang97@163.com; 13057588879@163.com; qsliu@nuist.edu.cn).

Digital Object Identifier 10.1109/TGRS.2022.3207551

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

However, this also brings unrelated information to small-scale objects, leading to incorrect results for them. On the contrary, small receptive fields in shallow networks may be enough for small-scale object segmentation, but they may only cover a small part of large-scale objects, producing inconsistent results for them. Therefore, how to deal with different scales of objects becomes a critical issue for high-quality HRSI segmentation.

There exist large amounts of modified FCNs attempting to address the above issue, and they often depend on one of the following three strategies: aggregating multiple convolution features from different layers, extracting multiscale features in one convolutional layer, and fusing multiscale information through multiple branches. A typical network for the first category of strategy is UNet [11], [12], [13], which adopts an encoder–decoder framework. The encoder consists of several convolution and downsampling layers to extract high-level semantic features, while the decoder is utilized to gradually upsample the extracted features and derive a segmentation result. During each upsampling process, skip-layer connections are employed to transfer the convolutional features from the encoder to the decoder. In [14], residual blocks were introduced to construct the encoder and the decoder, thus alleviating the problems of vanishing and exploding gradients. In [15], [16], [17], [18], attention modules were designed to fine-tune the convolutional features in the encoder before transferring them to the decoder. Different from traditional UNet models which only transfer one convolutional feature each time, RefineNet uses all convolutional layers as inputs to the decoder [19], [20]. Except for applying low-level features in the encoder to restore spatial information, up-pooling with recorded max-pooling indexes in the encoder could also be used to realize a similar purpose, which is the core idea of SegNet [21], [22]. Very recently, Zhao *et al.* proposed utilizing low-level features and recorded pooling indexes in different layers of the decoder to restore the spatial details [23].

For the second category of strategy, there also exist many works exploring different methods to extract multiscale features in one layer. An intuitive method is utilizing multiple convolution kernels with different sizes in parallel. For instance, Liu *et al.* [24] applied inception modules [25] to achieve this goal. Each module consists of four different combinations of convolutions. For large-scale objects which require large receptive fields, it is inefficient to adopt such modules, because the parameters and computational cost increase exponentially to the convolutional kernel size. Differently, pyramid pooling modules (PPMs) [26], first proposed in [27], use four different sizes of pooling operators to capture varied receptive fields. After each pooling operator, a convolutional layer is further employed to extract features. These convolutional features are finally upsampled to the same size and concatenated together. Although PPMs are more efficient than inception modules, they drop much spatial detail information, which is very important in HRSI segmentation, due to the use of pooling operators. Atrous spatial pyramid pooling (ASPP) module [28], [29] is an alternative method to overcome this drawback. It replaces the combination of pooling and traditional convolution with atrous convolution with different dilation rates. Upon ASPP, some works did modifications to make it more suitable to HRSIs [13], [30]. Take [13] as an instance, it proposed a cascaded way to aggregate convolutional features from global to local, instead of simply concatenating them together.

In contrast to the first two categories of strategies, the last category of strategy takes advantage of multiple branches to fuse multiscale information. Image pyramid is a representative method in this direction [31], [32]. It first resizes images into different scales and then inputs them into multiple parallel networks. Finally, the extracted multiscale features or segmentation results are fused together. Such a method costs a lot of GPU memory and training time. To save computation costs, different-scale images are often fed into a shared network. Due to the existence of stride convolution or pooling operators, convolutional features in different layers can be considered as multiscale representations for input images. Motivated by it, some works fed different-layer features into networks to get segmentation results [9], [33]. In [33], Zhou *et al.* took the last three block features in ResNet [34] as inputs and designed feature decoupling modules to embed cooccurrence relations into them. Similar to [9], multiple segmentation results were separately derived from these enhanced features and combined in a weighted summation manner. Another interesting work is [35], which followed the encoder–decoder framework to construct three networks. They comprised different sizes of convolutional kernels, thus multiscale results could be obtained. Recently, HRNet [36] has been explored in the field of HRSI segmentation [37], [38]. It started from one network, and gradually added other networks one by one to form multiple branches in parallel. The final output features in different branches were concatenated together to capture multiscale information.

To sum up, the first two classes of strategies attempt to construct a network which is able to simultaneously segment multiscale objects. However, the learning ability of a single network is limited, so such models often have to make tradeoffs between different-scale objects, leading to unsatisfactory segmentation results, especially for the small- or large-scale objects. The third class of strategies is a potential way to alleviate this issue by taking advantage of multiple subnetworks. In comparison with an individual network, the learning ability of multiple networks is more powerful if fully exploited. Nevertheless, there still exist some problems in designing and fusing different networks. In this article, we try to make a step forward along these directions. Specifically, we propose a multiscale progressive segmentation network to address the issue caused by objects with large-scale variations. Our network is mainly comprised of three subnetworks, and they are expected to be responsible for different-scale objects. To this end, we design variant network structures and scale guidance modules between neighboring subnetworks. At the segmentation phase, we construct a position-sensitive module to sufficiently consider the contributions of different subnetworks. In order to demonstrate the effectiveness of our proposed model, we conduct ablation studies on the proposed network and compare it with several state-of-the-art segmentation models using two benchmark datasets.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

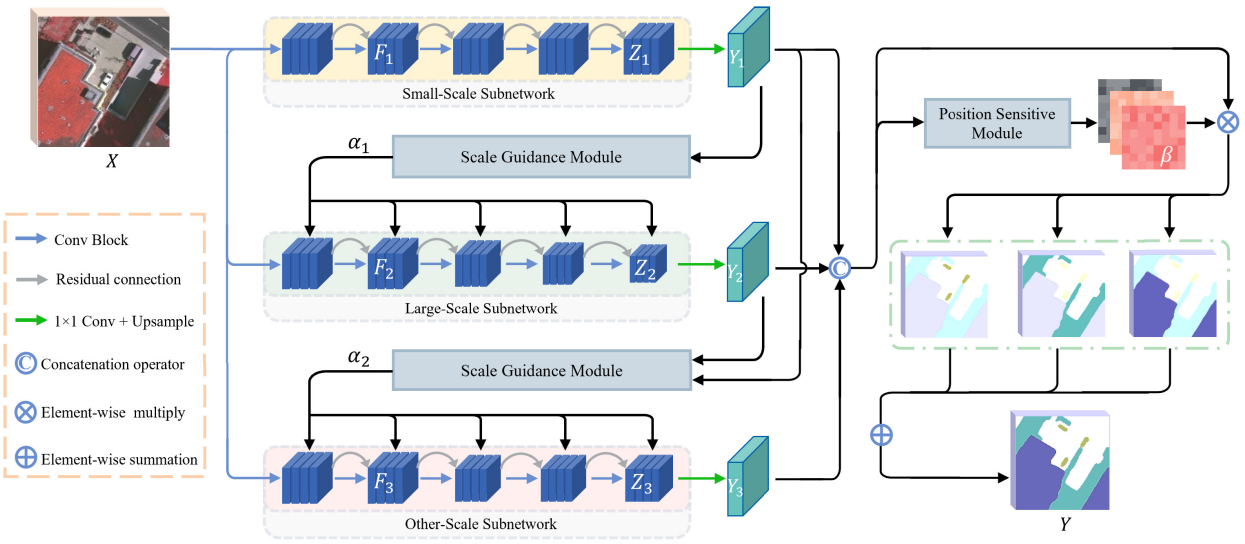


Fig. 1. Flowchart of our proposed multiscale progressive segmentation network.

The primary contributions of our article can be summarized as follows.

1) Different from most existing models, which attempt to segment all objects in an HRSI simultaneously, we propose a new framework with multiple subnetworks to gradually segment different-scale objects.
2) To make each subnetwork responsible for the specific scale object, we design a scale guidance module from the preceding subnetwork for guiding the feature learning process of the succeeding one.
3) In order to sufficiently combine the segmentation results derived from different subnetworks, we devise a position-sensitive module, which adaptively learns the importance of them at each position.

The remaining sections are organized as follows. In Section II, the details of the proposed model are introduced. In Section III, experimental data and results are described. Finally, the conclusions of this article are summarized in Section IV.

II. Methodology

A. Overall Framework

As shown in Fig. 1, a multiscale progressive segmentation network is proposed to deal with the challenge of large-scale variations in HRSIs. For an input HRSI $X \in \Re^{H\times W\times B}$, where H, W, and B represent the height, width, and channel numbers of X, respectively, it is sequentially fed into the small-scale subnetwork, the large-scale subnetwork, and the other-scale subnetwork. First, the small-scale subnetwork, which has relatively small receptive fields, puts focus on segmenting small-scale objects in X. Then, based on the segmentation result, a scale guidance module is constructed, and used to help the large-scale subnetwork segment large-scale objects in X. After that, another scale guidance module is built upon the segmentation results of the small-scale and large-scale subnetworks. Such a module is able to assist the other-scale subnetwork concentrate on dealing with the other scale objects. Finally, the outputs of the three subnetworks are combined together, and fed into a position-sensitive module to derive the final segmentation result.

B. Structure of Three Subnetworks

In an HRSI, there often exist different-scale objects, and the small- and large-scale objects are generally more difficult to segment than the other scale objects. Therefore, we specially design two subnetworks to segment the small- and large-scale objects, and another subnetwork to deal with the other ones. As illustrated in Fig. 1, each subnetwork is comprised of five convolutional blocks. Note that there exist residual connections between neighboring blocks to reuse the shallow features. Table I demonstrates the detailed configurations of different convolution blocks. Take the first row as an instance, the operator '[Conv7 $\times$ 7] $\times$ 2' means that there are two convolutional layers and each of them has a kernel size of 7 $\times$ 7. The corresponding stride '[2, 2]' in the same row indicates that the stride of each convolutional operator is 2.

For the small-scale subnetwork, the first convolutional block, named 'Block0' in Table I, contains two convolutional layers, whose kernel size is 7 $\times$ 7. The number of convolutional kernels and stride of each convolution are chosen as C and 2, respectively. In this article, C is empirically set to 64. For the rest of the convolutional blocks, we adopt different structures. Specifically, four successive convolutional layers, whose kernel size is 3 $\times$ 3 and kernel number is C, are deployed in each block. In order to preserve the spatial information of small-scale objects, the stride of each convolutional operator is set to 1. Through the five convolutional blocks, a feature representation $Z_1 \in \Re^{(H/4)\times(W/4)\times C}$ is obtained. Such representation is then fed into a 1 $\times$ 1 convolutional layer and an upsampling layer to derive a segmentation result $Y_1 \in \Re^{H\times W\times NC}$, where NC represents the number of land-cover classes to segment.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILED CONFIGURATIONS OF THREE SUBNETWORKS

Subnetwork	Layer	Operator	Stride	Output Dimension
Small-Scale subnetwork	Block0	$[Conv7\times7]\times2$	$[2,2]$	$H/4\times W/4\times C$
	Block1	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
	Block2	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
	Block3	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
	Block4	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
Large-Scale subnetwork	Block0	$[Conv7\times7]\times2$	$[2,2]$	$H/4\times W/4\times C$
	Block1	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
	Block2	$[Conv3\times3]\times4$	$[2,1,1,1]$	$H/8\times W/8\times 2C$
	Block3	$[Conv3\times3]\times4$	$[2,1,1,1]$	$H/16\times W/16\times 4C$
	Block4	$[Conv3\times3]\times4$	$[2,1,1,1]$	$H/32\times W/32\times 8C$
Other-Scale subnetwork	Block0	$[Conv7\times7]\times2$	$[2,2]$	$H/4\times W/4\times C$
	Block1	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/4\times W/4\times C$
	Block2	$[Conv3\times3]\times4$	$[2,1,1,1]$	$H/8\times W/8\times 2C$
	Block3	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/8\times W/8\times 2C$
	Block4	$[Conv3\times3]\times4$	$[1,1,1,1]$	$H/8\times W/8\times 2C$

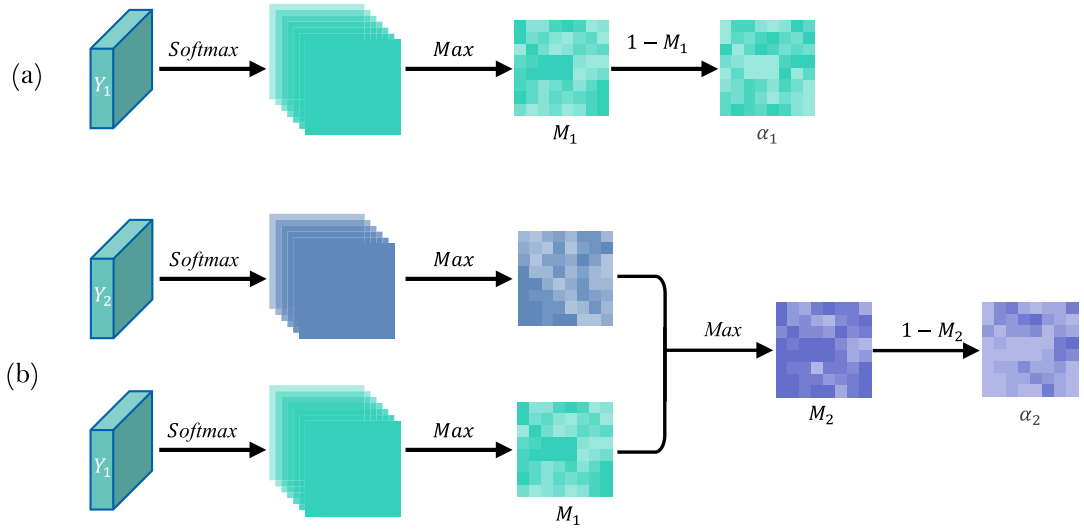


Fig. 2. Flowchart of the two-scale guidance modules. (a) First scale guidance module between the small-scale and the large-scale subnetworks. (b) Second scale guidance module between the large-scale and the other-scale subnetworks.

For the large-scale subnetwork, it has the same convolutional operators in the first two blocks (i.e., Block 0 and Block 1) as the small-scale subnetwork. Considering that the purpose of the large-scale subnetwork is extracting features of large-scale objects, the last three blocks are different from those of the small-scale subnetwork. In particular, each of them contains four convolutional layers, and the first one has a stride of 2 to increase receptive fields. The number of convolutional kernels from Block 2 to Block 4 are sequentially set to $2C$, $4C$, and $8C$. More importantly, for each of the five convolutional blocks, a scale guidance module, whose input is Y_1, is employed to refine the extracted features so that the discriminative information for large-scale objects is enhanced. Again, a 1×1 convolutional layer and an upsampling layer are used to achieve another segmentation result $Y_2\in\Re^{H\times W\times NC}$.

In comparison with the large-scale subnetwork, the other-scale subnetwork has the same operators in the first three blocks. For the last two blocks, it adopts four convolutional layers with a stride of 1 and kernel numbers of $2C$. Thus, a feature representation $Z_3\in\Re^{(H/8)\times(W/8)\times2C}$ is generated. In order to make each block learn powerful features for other-scale objects, another scale guidance module is designed, whose inputs are Y_1 and Y_2. Based on Z_3, a segmentation result $Y_3\in\Re^{H\times W\times NC}$ can be achieved via a 1×1 convolutional operator and an upsampling operator.

C. Scale Guidance Module

In order to make the large-scale and the other-scale subnetworks work as expected, two scale guidance modules are designed. Fig. 2 shows the flowchart of these two modules. Assuming the small-scale subnetwork performs well, its output Y_1 could act as an indicator to find the positions of small-scale objects. Since the large-scale subnetwork has a contrary

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

purpose in comparison with the small-scale subnetwork, it should pay less attention to the positions where the small-scale objects exist. Therefore, we can use Y_1 to construct the first scale guidance module, shown in Fig. 2(a). First, we adopt softmax to normalize Y_1. Then, for each spatial position in the normalized Y_1, we attempt to find its maximum value along NC channels. Such a value indicates the probability it belongs to small-scale objects. Finally, a guidance map α_1 is derived upon it. The whole process can be formulated as follows:

$$\alpha_1 = 1 - \text{Max}(\text{Softmax}(Y_1)) \tag{1}$$

where $\text{Max}(\cdot)$ is a function to calculate the maximum value at each position of an input tensor. After obtaining α_1, we employ it to refine the output of each convolutional block in the large-scale subnetwork. Take the second block output $F_2 \in \Re^{(H/4)\times(W/4)\times C}$ as an example, it is defined as

$$F_2 \leftarrow F_2 \otimes \Gamma(\alpha_1) \tag{2}$$

where "$\otimes$" denotes element-wise multiplication, and "$\Gamma(\cdot)$" represents a preprocessing operator, which is comprised of two steps. The first one is downsampling α_1 to make it have the same spatial size as that of F_2. The second one is replicating α_1 along the channel dimension to acquire the same number of channels as that of F_2.

Different from the first scale guidance module, the second scale guidance module takes both Y_1 and Y_2 as inputs, because it aims at helping the other-scale subnetwork segment other-scale objects except for the small- and large-scale ones. Fig. 2(b) shows the flowchart for deriving the guidance map α_2 of the second scale guidance module, which is formulated as

$$\alpha_2 = 1 - \widetilde{\text{Max}}(\text{Max}(\text{Softmax}(Y_1)), \text{Max}(\text{Softmax}(Y_2))) \tag{3}$$

where $\widetilde{\text{Max}}(\cdot,\cdot)$ denotes a function to find a larger value at each element from two input matrices. When α_2 is derived, it will be applied to fine-tune the extracted features from each convolutional block in the other-scale subnetwork. Suppose that the output of the second block is $F_3 \in \Re^{(H/4)\times(W/4)\times C}$, this value will be fine-tuned as

$$F_3 \leftarrow F_3 \otimes \Gamma(\alpha_2). \tag{4}$$

D. Position Sensitive Module

With the aid of the two-scale guidance modules, the three subnetworks succeed in segmenting different-scale objects. What remains to be accomplished is how to effectively aggregate three-scale segmentation results Y_1, Y_2, and Y_3. Most of existing methods adopt the "Sum" and "Concat" strategies [9], [11], [27] to fuse different segmentation results. For the first strategy, segmentation results are aggregated via an element-wise summation method. For the second category, they are concatenated along the channel dimension and then fed into a 1×1 convolutional layer to get the final result. Nevertheless, the above methods fail to consider the contributions of different-scale results in segmenting each object.

In this article, we argue that Y_1, Y_2, and Y_3 should have different contributions at each position according to the scale of objects to be segmented. To this end, we propose a position-sensitive module, aiming at adaptively learning three contribution maps, each of which corresponds to a specific-scale segmentation result. Fig. 3 shows the flowchart of the proposed position-sensitive module. It first combines Y_1, Y_2, and Y_3 together in a concatenation way, and then uses two convolutional layers to generate the contribution maps. Such a process can be expressed as

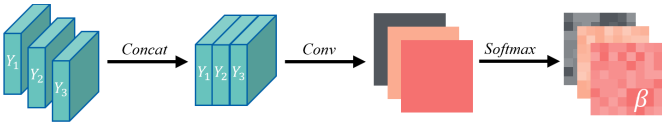


Fig. 3. Flowchart of the position sensitive module.

$$\beta = \text{Softmax}(f_2(f_1([Y_1; Y_2; Y_3]))) \tag{5}$$

where $\beta \in \Re^{H\times W\times 3}$ represents the generated contribution map, "[ ]" denotes a concatenating operator along the channel dimension, f_1 is a convolutional operator whose kernel size is 3×3 and kernel number is NC, and f_2 is a convolutional operator whose kernel size is 3×3 and kernel number is 3.

Assume $\beta_i, i \in \{1, 2, 3\}$ denotes the ith channel of β, its values describe the contribution of Y_i to the final segmentation result Y. Therefore, Y can be calculated by

$$Y = \sum_{i=1}^{3} \text{Rep}(\beta_i) \otimes Y_i \tag{6}$$

where $\text{Rep}(\cdot)$ means replicating the input matrix NC times along the channel dimension.

E. Loss Function

The proposed network is trained in an end-to-end manner. For each training sample, we can input it into the network and output four prediction results, including the three intermediate ones (i.e., Y_1, Y_2, and Y_3) and the final one (i.e., Y). In order to make the three subnetworks and the position-sensitive module learn as expected, we adopt the following combination loss based on these four results:

$$\tilde{L} = \frac{1}{3}\sum_{j=1}^{N}\sum_{i=1}^{3}\mathcal{L}\left(T^{(j)}, Y_i^{(j)}\right) + \lambda\sum_{j=1}^{N}\mathcal{L}\left(T^{(j)}, Y^{(j)}\right) \tag{7}$$

where $\tilde{L}$ is the loss value computed from the whole training samples, N is the number of training samples, $\mathcal{L}(\cdot,\cdot)$ is the cross-entropy loss function, $T^{(j)}$ is the label for the jth training sample, $Y^{(j)}$ is the final prediction result for the jth training sample, $Y_i^{(j)}$ is the ith intermediate result for the jth training sample, and λ is a hyperparameter to balance the effects of two loss functions.

III. Experiments

In this section, two public datasets from the ISPRS[1] are used to test the performance of our proposed network. First of all, several preprocessing methods about the datasets and

[1]http://www2.isprs.org/commissions/comm3/wg4/semantic-labeling.html

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

implementation details about the network are introduced. Then, ablation studies on the scale guidance modules and the position-sensitive module are conducted. Finally, the proposed network is compared with several state-of-the-art deep methods and benchmark models on the datasets.

A. Data Description

1) Vaihingen Dataset: This dataset contains 33 true orthophoto tiles with an average size of 2494 × 2064 pixels and a ground sampling distance (GSD) of 9 cm. There are three channels in each tile, containing near-infrared, red, and green. In addition, the corresponding digital surface model (DSM) is provided on the official website. All tiles are labeled from six classes, including *impervious surfaces*, *building*, *low vegetation*, *tree*, *car*, and *background*. The same as the setting by Commission III of the ISPRS,[2] we use 16 images for training and 17 images for testing, respectively.

2) Potsdam Dataset: It contains 38 true orthophoto tiles with a GSD of 5 cm. All tiles have the same size of 6000 × 6000 pixels. There exist three channels in each tile, including near-infrared, red, and green. Different from the Vaihingen dataset, normalized DSM (nDSM) is also available for each tile in addition to DSM. All tiles are also labeled from the same six classes as that of the Vaihingen dataset. Following the setting by Commission III of the ISPRS,[3] we used 24 images for training and 14 images for testing.

B. Data Preprocessing

As presented above, there only exist tens of training images on both datasets. Directly using them to train our network easily leads to the overfitting problem. To address this issue, we adopt several strategies to augment the training set, including patch cropping, appearance transformation, image rotation, and scaling. Some explanations about them are as follows.

1) Patch Cropping: For each training image, a 256 × 256 sliding window with a stride of 64 is used to crop image patches from it. In order to make full use of the image boundary information, each image is filled with a reflection method. During the testing phase, the same cropping and filling operators are applied to each test image, and the averaged prediction results are computed for the overlapping areas.

2) Image Rotation and Scaling: After patch cropping, we randomly employ vertical and horizontal flipping operations to each image patch with a probability of 0.25. Besides, we also randomly rotate image patches from 0 to 45°, and resize them between 0.5 and 2 times with a probability of 0.25.

3) Appearance Transformation: The brightness, saturation, contrast, and sharpness of the image patches are transformed randomly with enhancement factors ranging from 0 to 3. Take the brightness transformation as an example, the image brightness will decrease when the enhancement factor is less than 1, while it will increase when the enhancement factor is greater than 1.

[2]http://www2.isprs.org/commissions/comm3/wg4/2d-sem-label-vaihingen.html

[3]http://www2.isprs.org/commissions/comm3/wg4/2d-sem-label-potsdam.html

TABLE II
ABLATION STUDY ON TWO DATASETS

Dataset	Method	OA	mIoU
Vaihingen	MPSegNet-S	91.2	80.4
	MPSegNet-P	91.4	80.0
	MPSegNet	91.6	80.5
Potsdam	MPSegNet-S	91.0	77.3
	MPSegNet-P	91.2	77.3
	MPSegNet	91.4	77.9

C. Implementation Details

Considering that the normalized difference vegetation index (NDVI) is an effective indicator for vegetation segmentation, we calculate it for each image as follows:

$$\mathrm{NDVI} = \frac{IR - R}{IR + R} \tag{8}$$

where IR and R denote the near-infrared and red channels, respectively. In addition, we used the nDSM provided by [39] on the Vaihingen dataset. Thus, a composite input image comprised of near-infrared, red, green, DSM, nDSM, and NDVI is constructed.

Three evaluation metrics are used to assess the performance of our proposed network, including overall accuracy (OA), F1 score, and mean intersection over union (mIoU). OA is the ratio of the number of correctly segmented pixels to the total number of pixels. F1 is calculated as the harmonic mean of precision and recall. mIoU represents the overlapping rate between the predicted segmentation map and the ground-truth segmentation map.

All the experiments are implemented by PyTorch framework and conducted on a 64-bit computer with 32-GB memory and a GPU of GeForce GTX TITAN X. To optimize the network parameters, we adopt a stochastic gradient descent optimizer with a momentum of 0.9 and a weight decay of 0.0005. The initial learning rate is set to 0.01, and the poly learning rate policy is used to update the learning rate after each iteration by multiplying $(1 - (\mathrm{iter}/\mathrm{max_iter}))^{0.9}$, where 'iter' and 'max_iter' represent the specific iteration times and the total number of iterations, respectively.

D. Analysis on the Proposed Network

1) Ablation Study: In the proposed network, the scale guidance modules and the position-sensitive module are two important components. To evaluate their effects on the segmentation performance, we conduct ablation studies on two datasets. Table II compares the experimental results with and without using these two components. For simplicity, our proposed network is abbreviated as MPSegNet. In Table II, MPSegNet-S and MPSegNet-P indicate eliminating the scale guidance modules and the position-sensitive module from MPSegNet, respectively. Compared to MPSegNet, MPSegNet-S decreases OA from 91.6 to 91.2 (0.4↓), and mIoU from 80.5 to 80.4 (0.1↓), while MPSegNet-P decreases OA from 91.6 to 91.2 (0.2↓), and mIoU from 80.5 to 80.0 (0.5↓) on the Vaihingen dataset. Similar conclusions can be observed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

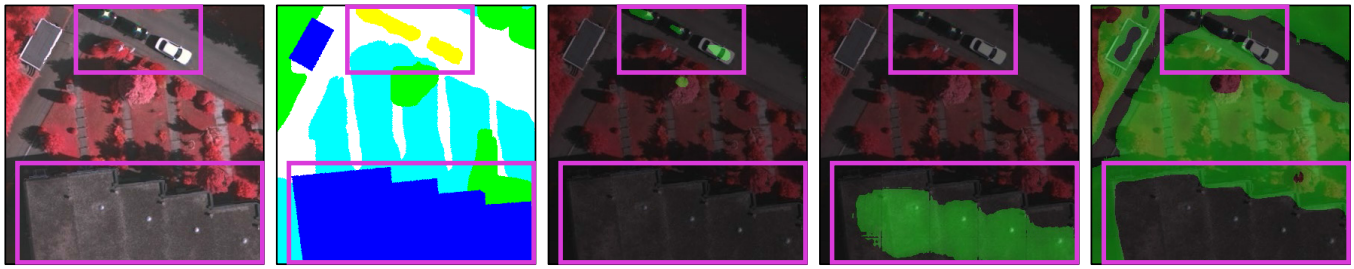


Fig. 4. Visualization of β in the position-sensitive module. (a) Input image. (b) Groundtruth map. (c) β_1 overlapped the input image. (d) β_2 overlapped the input image. (e) β_3 overlapped the input image.

TABLE III
COMPARISONS WITH STATE-OF-THE-ART DEEP MODELS ON THE VAIHINGEN DATASET

Method	Impervious Surfaces	Building	Low Vegetation	Tree	Car	OA	Mean F1	mIoU
FCN-8s [9]	90.7	94.3	80.5	88.0	79.7	89.6	86.6	77.0
UNet [11]	90.7	94.5	80.8	88.2	82.9	89.9	87.4	78.3
PSPNet [27]	91.2	94.8	81.5	88.5	77.2	90.2	86.6	77.0
DeepLabv3+ [40]	91.4	95.1	81.2	88.3	83.9	90.3	88.0	79.0
CGFDN [33]	91.8	94.7	83.1	**89.7**	82.9	91.0	88.4	79.4
MPSegNet	**92.4**	**95.7**	**83.4**	89.5	**84.6**	**91.6**	**89.1**	**80.5**

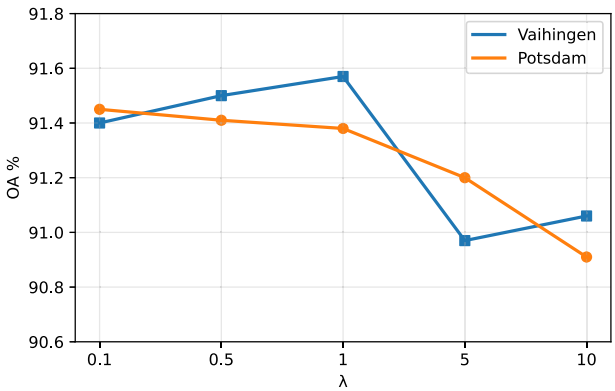


Fig. 5. Performance of MPSegNet with different λ values.

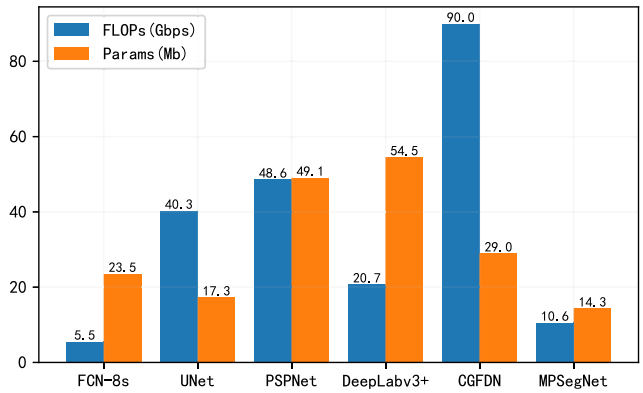


Fig. 6. Computational complexity and model size of different deep learning models.

in the Potsdam dataset. Specifically, MPSegNet-S reduces OA and mIoU by 0.4 and 0.6, respectively, while MPSegNet-P reduces OA and mIoU by 0.2 and 0.6, respectively. Fig. 4 takes an image as an instance, and shows the results of β in the position-sensitive module. The positions with green colors in Fig. 4(c)–(e) denote larger contributions to Y than the other positions. From this figure, it is obvious to see that β_1, β_2, and β_3 make relatively more contributions to small-, large-, and other-scale objects, respectively. These conclusions validate the effectiveness of the proposed two components.

2) Hyperparameter Selection: As shown in (7), λ is a vital hyperparameter to choose. Here, we change its value from a candidate set $\{0.1, 0.5, 1, 5, 10\}$ and evaluate the corresponding segmentation performance. Fig. 5 demonstrates OAs achieved by using different λ values on the two datasets. It can be observed that on the Potsdam dataset, OA values gradually decrease as λ increases. Therefore, we can set the optimal λ value to 0.1. Differently, on the Vaihingen dataset, OA values first increase when λ changes from 0.1 to 1, and then decrease from 1 to 10. The highest OA value appears when λ equals 1. So, the optimal λ value is chosen as 1 on the Vaihingen dataset.

E. Compared With State-of-the-Art Deep Models

In this section, we compare the proposed MPSegNet with several state-of-the-art deep models, including FCN-8s [9], UNet [11], PSPNet [27], DeepLabv3+ [40], and CGFDN [33]. For a fair comparison, these networks adopt the same structure as that proposed in the original articles, and they also use the composite image, consisting of near-infrared, red, green, DSM, nDSM, and NDVI, as inputs.

Tables III and IV report the detailed segmentation performance achieved by different models on the Vaihingen dataset and the Potsdam dataset, respectively. As can be observed, FCN-8s achieves inferior performance than the other models

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISONS WITH STATE-OF-THE-ART DEEP MODELS ON THE POTSDAM DATASET

Method	Impervious Surfaces	Building	Low Vegetation	Tree	Car	OA	Mean F1	mIoU
FCN-8s [9]	91.6	96.2	84.7	86.3	94.4	89.6	90.6	74.7
UNet [11]	92.0	96.0	84.8	86.2	95.4	89.6	90.9	75.3
PSPNet [27]	91.9	96.6	85.4	85.8	95.1	89.9	91.0	75.4
DeepLabv3+ [40]	92.2	96.5	85.3	87.3	95.6	90.3	91.4	76.8
CGFDN [33]	92.6	97.0	86.8	87.6	94.9	90.8	91.8	76.3
MPSegNet	**93.2**	**97.5**	**86.9**	**88.3**	**96.2**	**91.4**	**92.4**	**77.9**

TABLE V
COMPARISONS WITH MORE BENCHMARK MODELS ON THE VAIHINGEN DATASET

Method	Impervious Surfaces	Building	Low Vegetation	Tree	Car	OA	Mean F1
RIT_L8 [41]	89.6	92.2	81.6	88.6	76.0	87.8	85.6
AZ1	89.6	92.4	78.6	87.7	78.2	87.3	85.3
UFMG_4 [42]	91.1	94.5	82.9	88.8	81.3	89.4	87.7
CVEO3 [12]	91.0	93.0	81.3	88.3	83.0	88.6	87.3
CAS_Y3 [26]	91.0	93.3	82.2	88.2	71.3	88.7	85.2
DST_2 [43]	90.5	93.7	**83.4**	89.2	72.6	89.1	85.9
MPSegNet	**92.4**	**95.7**	**83.4**	**89.5**	**84.6**	**91.6**	**89.1**

TABLE VI
COMPARISONS WITH MORE BENCHMARK MODELS ON THE POTSDAM DATASET

Method	Impervious Surfaces	Building	Low Vegetation	Tree	Car	OA	Mean F1
RIT_L7 [41]	91.2	94.6	85.1	85.1	92.8	88.4	89.8
AZ3	93.1	96.3	**87.2**	**88.6**	96.0	90.7	92.2
UFMG_4 [42]	90.8	95.6	84.4	84.3	92.4	87.9	89.5
CVEO2 [12]	91.3	94.5	86.5	87.7	95.1	89.0	91.0
CAS_Y3 [26]	92.2	95.7	**87.2**	87.6	95.6	90.1	91.7
DST_5 [43]	92.5	96.4	86.7	88.0	94.7	90.3	91.7
MPSegNet	**93.2**	**97.5**	86.9	88.3	**96.2**	**91.4**	**92.4**

in terms of OA, mean F1, and mIoU. By employing skip-layer connections, UNet combines features from different convolutional layers, thus improving the segmentation performance. Take the Vaihingen dataset as an instance, UNet improves OA, mean F1, and mIoU by 0.3%, 0.8%, and 1.3%, respectively, in comparison with FCN-8s. However, the multiscale features extracted by UNet are limited by the fixed receptive field of the convolutional kernels. PSPNet and DeepLabv3+ which utilize multiple convolution kernels with different sizes could alleviate this issue and improve the segmentation performance of UNet in most cases. Compared to the above models, CGFDN is able to obtain the largest OA and mean F1 values on both datasets. More importantly, our proposed MPSegNet can further improve the segmentation performance significantly. Specifically, the OA and mean F1 values are improved by more than 0.6% on both datasets; the mIoU value is boosted by more than 1% on both datasets. In terms of each class performance, our proposed MPSegNet gets the highest values in 4 of 5 classes on the Vaihingen dataset and all five classes on the Potsdam dataset. Besides, we compare the computational complexity and model size of different models. As shown in Fig. 6, our proposed MPSegNet is equipped with the smallest number of parameters in all models. Although FCN-8s exists the lowest floating-point operations per second (FLOPs), it has significantly more numbers of parameters than our model. All the above observations can validate the effectiveness of our proposed MPSegNet.

In addition to the quantitative comparisons, Figs. 7 and 8 also visualize some segmentation results on the two datasets. In each figure, two images are chosen to demonstrate. Different colors represent different classes of objects, and purple boxes show some typical objects to segment. For the first image in both figures, one typical object is the large *building*, and another one is a small object, such as the *impervious surface* in Fig. 7 and the *background* in Fig. 8. It can be observed that the large *building* in both datasets is partially segmented as *low vegetation* or *background*, while the small *impervious surface* and the *background* are misclassified as *building* or *car* by all comparison models. For the second image in both figures, there are many similar sizes of objects to segment, and the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

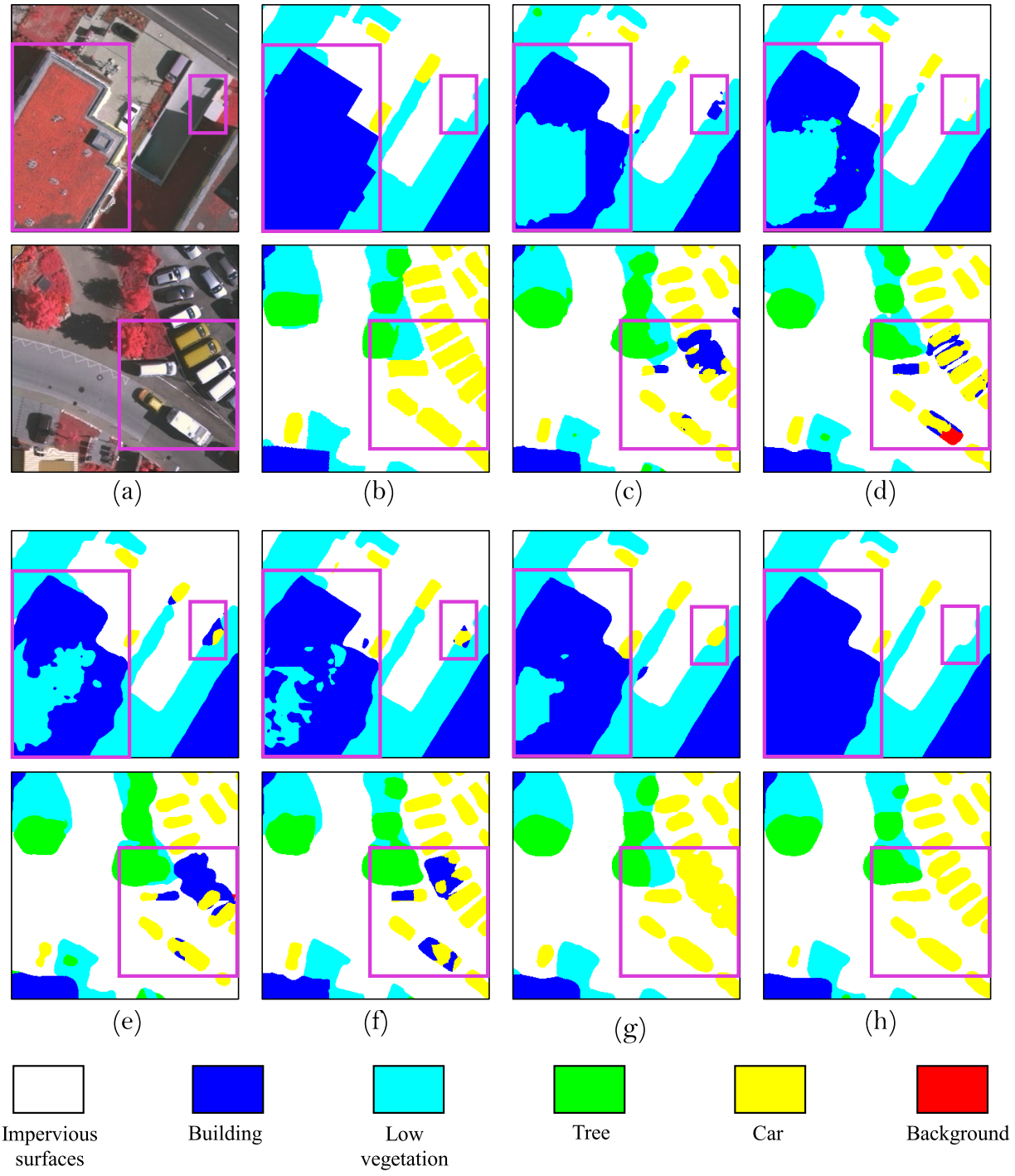


Fig. 7. Segmentation maps achieved by different models on the Vaihingen dataset. (a) Input image. (b) Groundtruth map. (c) FCN-8s. (d) UNet. (e) PSPNet. (f) DeepLabv3+. (g) CGFDN. (h) MPSegNet.

typical object is *car*. Most of the comparison models easily misclassify the *car* as *background* or *building*. Differently, our proposed MPSegNet is able to segment all the typical objects in both images accurately, which further demonstrates its effectiveness.

F. Compared With More Benchmark Models

In the past few years, a lot of segmentation models have tested their performance on the Vaihingen and Potsdam datasets. To further verify the performance of our proposed MPSegNet, we choose some typical deep learning-related models to compare, including RIT_L8 (RIT_L7) [41], AZ1 (AZ3), UFMG_4 [42], CVOE_3 (CVEO2) [12], CAS_Y3 [26], and DST_2 (DST_5) [43]. Since all of them adopt the same experimental settings as ours, it is fair and convenient to directly refer to their published results.

Tables V and VI demonstrate the comparison results in terms of each class performance, OA, and mean F1. On the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

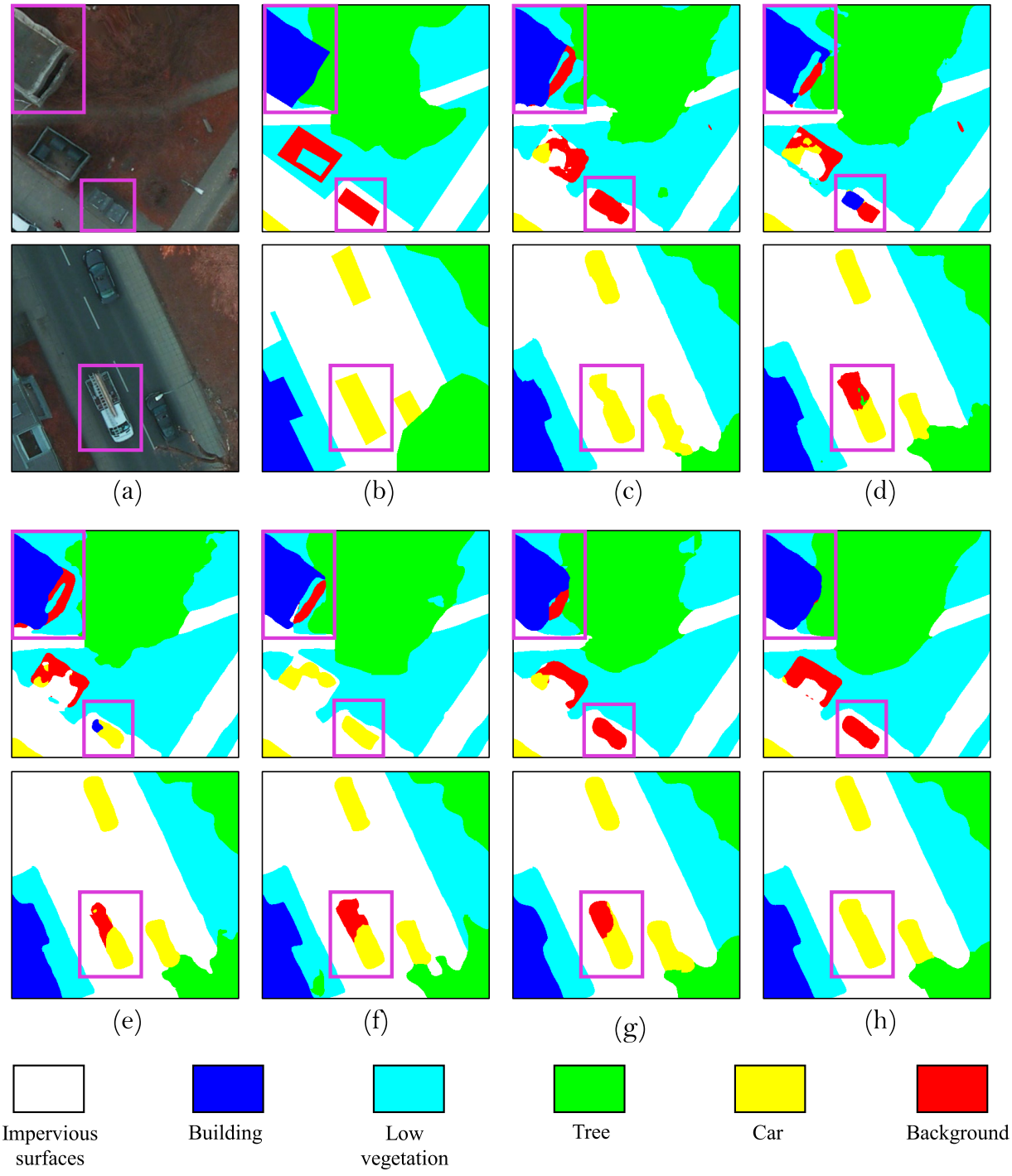


Fig. 8. Segmentation maps achieved by different models on the Potsdam dataset. (a) Input image. (b) Groundtruth map. (c) FCN-8s. (d) UNet. (e) PSPNet. (f) DeepLabv3+. (g) CGFDN. (h) MPSegNet.

Vaihingen dataset, UFMG_4 obtains the second highest OA and mean F1 values. Compared with it, our proposed MPSegNet is capable of improving the segmentation accuracy in all five classes, so the OA and mean F1 values are significantly improved from 89.4 to 91.6, and from 87.7 to 89.1, respectively. On the Potsdam dataset, UFMG_4 works not good as other models. The second best model is AZ3, due to the outstanding performance in the *low vegetation* and *tree* classes. Nevertheless, our proposed MPSegNet can achieve similar performance in these two classes. For example, in the *low vegetation* class, MPSegNet achieves an accuracy of 86.9, which is close to 87.2. Besides these two classes, MPSegNet obtains the best results in the other three classes, leading to the highest OA and mean F1 values.

IV. Conclusion

In this article, a multiscale progressive segmentation network was proposed to address the issue of simultaneously segmenting objects with large-scale variations in HRSIs. Different from most existing models, our proposed network

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

employed three subnetworks and scale guidance modules for separately segmenting objects with small, large, and other scales. In addition, a position-sensitive module was designed to sufficiently consider the contributions of different subnetworks when combining the three segmentation results. To evaluate the segmentation performance of our proposed network, we conducted experiments on two benchmark datasets (i.e., Vaihingen and Potsdam) and compared with several segmentation models. Additionally, we also analyzed the effects of different components and hyperparameters in the proposed model.

REFERENCES

[1] V. Mnih and G. E. Hinton, "Learning to detect roads in high-resolution aerial images," in *Proc. Eur. Conf. Comput. Vis.* Berlin, Germany: Springer, 2010, pp. 210–223.

[2] N. Audebert, B. Le Saux, and S. Lefèvre, "Semantic segmentation of earth observation data using multimodal and multi-scale deep networks," in *Proc. Asian Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 180–196.

[3] M. Kampffmeyer, A.-B. Salberg, and R. Jenssen, "Semantic segmentation of small objects and modeling of uncertainty in urban remote sensing images using deep convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2016, pp. 1–9.

[4] Y. Bengio, I. Goodfellow, and A. Courville, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[5] S. Paisitkriangkrai, J. Sherrah, P. Janney, and A. van den Hengel, "Semantic labeling of aerial and satellite imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 7, pp. 2868–2881, Jul. 2016.

[6] G. Li, L. Li, H. Zhu, X. Liu, and L. Jiao, "Adaptive multiscale deep fusion residual network for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 11, pp. 8506–8521, Nov. 2019.

[7] S. Jia *et al.*, "A lightweight convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4150–4163, May 2021.

[8] S. Jia *et al.*, "A semisupervised Siamese network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2022.

[9] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 3431–3440.

[10] M. Volpi and D. Tuia, "Dense semantic labeling of subdecimeter resolution images with convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 881–893, Feb. 2016.

[11] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput.-Assist. Intervent*. Cham, Switzerland: Springer, 2015, pp. 234–241.

[12] G. Chen, X. Zhang, Q. Wang, F. Dai, Y. Gong, and K. Zhu, "Symmetrical dense-shortcut deep fully convolutional networks for semantic segmentation of very-high-resolution remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 5, pp. 1633–1644, May 2018.

[13] Y. Liu, B. Fan, L. Wang, J. Bai, S. Xiang, and C. Pan, "Semantic labeling in very high resolution images via a self-cascaded convolutional neural network," *ISPRS J. Photogram. Remote Sens.*, vol. 145, pp. 78–95, Nov. 2018.

[14] F. I. Diakogiannis, F. Waldner, P. Caccetta, and C. Wu, "ResUNet-A: A deep learning framework for semantic segmentation of remotely sensed data," *ISPRS J. Photogramm. Remote Sens.*, vol. 162, pp. 94–114, Apr. 2020.

[15] R. Shang, J. Zhang, L. Jiao, Y. Li, N. Marturi, and R. Stolkin, "Multi-scale adaptive feature fusion network for semantic segmentation in remote sensing images," *Remote Sens.*, vol. 12, no. 5, p. 872, Mar. 2020.

[16] R. Li *et al.*, "Multiattention network for semantic segmentation of fine-resolution remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2021.

[17] R. Liu, L. Mi, and Z. Chen, "AFNet: Adaptive fusion network for remote sensing image semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7871–7886, Sep. 2020.

[18] X. Yang *et al.*, "An attention-fused network for semantic segmentation of very-high-resolution remote sensing imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 177, pp. 238–262, Jul. 2021.

[19] G. Lin, A. Milan, C. Shen, and I. Reid, "RefineNet: Multi-path refinement networks for high-resolution semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1925–1934.

[20] R. Kemker, C. Salvaggio, and C. Kanan, "Algorithms for semantic segmentation of multispectral remote sensing imagery using deep learning," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 60–77, Nov. 2018.

[21] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder–decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Jan. 2017.

[22] Y. Sun, Y. Tian, and Y. Xu, "Problems of encoder–decoder frameworks for high-resolution remote sensing image segmentation: Structural stereotype and insufficient learning," *Neurocomputing*, vol. 330, pp. 297–304, Feb. 2019.

[23] Q. Zhao, J. Liu, Y. Li, and H. Zhang, "Semantic segmentation with attention mechanism for remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2021.

[24] Y. Liu, D. M. Nguyen, N. Deligiannis, W. Ding, and A. Munteanu, "Hourglass-shapenetwork based semantic segmentation for high resolution aerial imagery," *Remote Sens.*, vol. 9, no. 6, p. 522, 2017.

[25] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

[26] B. Yu, L. Yang, and F. Chen, "Semantic segmentation for high spatial resolution remote sensing images based on convolution neural network and pyramid pooling module," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 9, pp. 3252–3261, Sep. 2018.

[27] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 2881–2890.

[28] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2017.

[29] L. Mi and Z. Chen, "Superpixel-enhanced deep neural forest for remote sensing image semantic segmentation," *ISPRS J. Photogramm. Remote Sens.*, vol. 159, pp. 140–152, Jan. 2020.

[30] X. Chen *et al.*, "Adaptive effective receptive field convolution for semantic segmentation of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 4, pp. 3532–3546, Apr. 2020.

[31] Q. Liu, R. Hang, H. Song, and Z. Li, "Learning multiscale deep features for high-resolution satellite image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 117–126, Jan. 2017.

[32] L.-C. Chen, Y. Yang, J. Wang, W. Xu, and A. L. Yuille, "Attention to scale: Scale-aware semantic image segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 3640–3649.

[33] F. Zhou, R. Hang, and Q. Liu, "Class-guided feature decoupling network for airborne image segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2245–2255, Mar. 2020.

[34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.

[35] Y. Sun, X. Zhang, Q. Xin, and J. Huang, "Developing a multi-filter convolutional neural network for semantic segmentation using high-resolution aerial imagery and LiDAR data," *ISPRS J. Photogramm. Remote Sens.*, vol. 143, pp. 3–14, Sep. 2018.

[36] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 5693–5703.

[37] J. Zhang, S. Lin, L. Ding, and L. Bruzzone, "Multi-scale context aggregation for semantic segmentation of remote sensing images," *Remote Sens.*, vol. 12, no. 4, p. 701, Feb. 2020.

[38] Z. Xu, W. Zhang, T. Zhang, and J. Li, "HRCNet: high-resolution context extraction network for semantic segmentation of remote sensing images," *Remote Sens.*, vol. 13, no. 1, p. 71, Dec. 2021.

[39] M. Gerke, "Use of the stair vision library within the ISPRS 2D semantic labeling benchmark (Vaihingen)," ITC, Univ. Twente, Enschede, The Netherlands, Tech. Rep., 2014, doi: 10.13140/2.1.5015.9683.

[40] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder–decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[41] S. Piramanayagam, E. Saber, W. Schwartzkopf, and F. Koehler, "Supervised classification of multisensor remotely sensed images using a deep learning framework," *Remote Sens.*, vol. 10, no. 9, p. 1429, Sep. 2018.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.

[42] K. Nogueira, M. D. Mura, J. Chanussot, W. R. Schwartz, and J. A. D. Santos, "Dynamic multicontext segmentation of remote sensing images based on convolutional networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7503–7520, Oct. 2019.

[43] J. Sherrah, "Fully convolutional networks for dense semantic labelling of high-resolution aerial imagery," 2016, *arXiv:1606.02585*.



Renlong Hang (Member, IEEE) received the Ph.D. degree from the School of Computer and Software, Nanjing University of Information Science and Technology, Nanjing, China, in 2017.

From 2018 to 2019, he was a Post-Doctoral Researcher with the Department of Computer Science and Electrical Engineering, University of Missouri-Kansas City, Kansas City, MO, USA. He has authored or coauthored over 30 peer-reviewed articles in international journals, such as the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, and the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS. His research interests include machine learning, pattern recognition, and their applications to remote sensing image processing.



Ping Yang (Graduate Student Member, IEEE) received the B.S. degree in electronic science and technology from the Hunan Institute of Engineering, Xiangtan, China, in 2020. He is currently pursuing the M.S. degree in control engineering with the School of Automation, Nanjing University of Information Science and Technology, Nanjing, China.

His research interests include deep learning and remote sensing image processing.



Feng Zhou (Member, IEEE) received the Ph.D. degree in information and communication engineering from the Nanjing University of Information Science and Technology, Nanjing, China, in 2021.

He is currently a Lecturer with the School of Information Engineering, Nanjing Audit University, Nanjing. His research interests include deep learning and remote sensing image processing.



Qingshan Liu (Senior Member, IEEE) received the M.S. degree from Southeast University, Nanjing, China, in 2000, and the Ph.D. degree from the Chinese Academy of Sciences, Beijing, China, in 2003.

From 2010 to 2011, he was an Assistant Research Professor with the Department of Computer Science, Computational Biomedicine Imaging and Modeling Center, Rutgers, The State University of New Jersey, Piscataway, NJ, USA. Before he joined Rutgers University, Piscataway, he was an Associate Professor with the National Laboratory of Pattern Recognition, Chinese Academy of Sciences. During June 2004 and April 2005, he was an Associate Researcher with the Multimedia Laboratory, The Chinese University of Hong Kong, Hong Kong. He is currently a Professor with the School of Computer and Software, Nanjing University of Information Science and Technology, Nanjing. His research interests include image and vision analysis.

Dr. Liu received the President Scholarship of the Chinese Academy of Sciences in 2003.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:42:57 UTC from IEEE Xplore. Restrictions apply.